Python Training Notes

From Zero to OOPs — A Personal Textbook

Day 1	Day 2	Day 3	Day 4	Day 5	Day 6

• Basics + Syntax	• Data Structures	• Modules + Exceptions
• OOPs	• Advanced Python	• Testing + Regex

A complete guide from your own training sessions
Airport Security Management System · Smart Library System · and more

Table of Contents

→ Composition & Aggregation

Day 5 Advanced Python

→ Collections Module

→ Regular Expressions

→ Decorators

→ Generators

→ Context Managers

→ Database Connectivity

→ Best Practices & PEP 8

Day 6 Testing

→ STLC & Testing Types

→ Unit Testing with unittest

→ Test Fixtures

→ Parameterized Tests

→ Airport Management System — Full OOPs Project

DAY 1

Python Basics

Introduction · Data Types · Variables · Control Flow · Strings

1.1 What is Python?

Python is a high-level, interpreted, general-purpose programming language created by **Guido van Rossum in 1991**. It was designed to reduce complexity, improve code readability, and increase developer productivity.

■ Where Python is Used Today

- Web Development — Instagram, Django, Flask
- AI / Machine Learning — ChatGPT, TensorFlow, PyTorch
- Data Science — Netflix recommendations, Pandas, NumPy
- Automation — Sending emails, scripting repetitive tasks
- Cybersecurity — Penetration testing, scanning tools
- DevOps — Deployment scripts, Ansible
- Cloud — AWS Lambda, Google Cloud Functions

Key Features

High-Level Language	Acts as a bridge between human and machine. Easy to code, debug, read, and learn.
Interpreted	Runs line-by-line — easy debugging, immediate feedback. But slightly slower than compiled languages.
Object-Oriented	Everything is an object. Supports code reusability, maintainability, scalability.
Dynamically Typed	You don't declare variable types — Python figures it out automatically.
Platform Independent	Same code runs on Windows, macOS, Linux — write once, run anywhere.
Rich Standard Library	Huge built-in toolkit — you don't reinvent the wheel.

How Python Code Runs

Flow of Execution

Common Errors

Error Type	Cause	Example
Syntax Error	Wrong Python grammar	Missing colon after if
Indentation Error	Wrong spacing/tabs	Code not aligned properly
Name Error	Variable doesn't exist or has no value	print(x) before x = 5
Type Error	Wrong type operation	print("Age: " + 25)

1.2 Python Syntax Basics

Input, Output & Comments

```
# Output
print("Hello, Python!")

# Input
name = input("Enter your name: ")

# Single-line comment (use # )

# Multi-line comment:
```

```
"""
This is a
multi-line comment
"""
```

1.3 Variables & Data Types

A **variable** is a name that refers to a value stored in memory. Python is dynamically typed — you never need to declare the type.

```
student_name = 'Alice' # str
student_age = 20 # int
cgpa = 8.75 # float
is_active = True # bool
scores = [85, 90, 78] # list
info = {'id': 101} # dict
```

Data Type Reference

Type	Example	Memory	Notes
int	age = 20	28 bytes	No fixed size limit
float	price = 9.99	24 bytes	15-17 decimal digits precision
complex	z = 5+6j	32 bytes	Imaginary numbers
str	"Hello"	Varies	Immutable
bool	True / False	28 bytes	True=1, False=0
list	[1, 2, 3]	56 bytes+	Mutable, ordered
tuple	(1, 2, 3)	40 bytes+	Immutable, ordered
set	{1, 2, 3}	~200 bytes	Unordered, unique
dict	{"a": 1}	240 bytes+	Key-value pairs

■ Typecasting Reminder

- input() ALWAYS returns a string.
- Use int(input()) to get an integer from the user.
- Use float(input()) for decimal numbers.

- Example: age = int(input("Enter age: "))

1.4 Operators

Category	Operators	Use
Arithmetic	+ - * / % // **	Maths — // is floor division, ** is power
Assignment	= += -= *= /=	Assign or update values
Comparison	== != > < >= <=	Compare values → returns True/False
Logical	and or not	Combine conditions
Membership	in not in	Check if item exists in sequence
Identity	is is not	Compare memory addresses
Bitwise	& | ^ ~ << >>	Binary-level operations

1.5 Control Flow

Conditional Statements

```
# if / elif / else
marks = int(input('Enter marks: '))

if marks >= 90:
print("Grade: A")
elif marks >= 75:
print("Grade: B")
elif marks >= 60:
print("Grade: C")
else:
print("Grade: F")
```

Loops

for loop

```
for i in range(1, 6):
print(i) # 1 2 3 4 5

languages = ["Py","Java","C++"]
for lang in languages:
print(lang)
```

while loop

```
count = 1
while count <= 5:
print(count)
count += 1

# No do-while in Python!
```

■ break vs continue

- break — exits the loop completely
- continue — skips the current iteration and goes to next

1.6 String Functions

Strings are immutable sequences of characters. Python gives you a massive toolkit of built-in methods.

```
text = "Library Management"

# Slicing
print(text[0:7]) # 'Library'
print(text[::-1]) # reversed string

# Case
print(text.upper()) # LIBRARY MANAGEMENT
print(text.lower()) # library management

# Search
print(text.find('z')) # -1 (not found)
print(text.startswith('Lib')) # True
print(text.endswith('ment')) # True

# Replace / Split / Join
print(text.replace("Library", "Digital"))
print("Hello World".split(" ")) # list
print(",".join(["Py","Java","C#"]))

# Check type
print("bhagya".isalpha()) # True
print("12345".isdigit()) # True
print("user1".isalnum()) # True

# Padding
```

```
print("25".zfill(5)) # 00025
print("hi".center(10,"*")) # ****hi****
```

■ String Task

taskString = " Python Programming "

1. Remove spaces from head and tail → .strip()
2. Convert all to uppercase → .upper()
3. Replace 'Python' with 'Java' → .replace()
4. Count how many times 'm' appears → .count()

1.7 Number & Date Functions

```
import math

print(round(9.876, 2)) # 9.88
print(abs(-42)) # 42
print(divmod(7, 3)) # (2, 1)
print(math.sqrt(16)) # 4.0
print(math.floor(4.9)) # 4

# Datetime
import datetime
print(datetime.date.today()) # 2026-06-01
print(datetime.datetime.now().strftime("%d-%B-%Y"))

# Parsing string to date
dob = "23-May-2026"
date_obj = datetime.datetime.strptime(dob, "%d-%b-%Y")
```

DAY 2

Data Structures

Lists · Dictionaries · Sets · Map/Filter/Reduce · Functions · Modules

2.1 Lists

A list is an **ordered, mutable** collection. You can store anything in it and change it anytime.

```
books = ["Python", "Java", "C++", "JavaScript"]

# Access
print(books[0]) # 'Python'
print(books[-1]) # 'JavaScript' (last)
print(books[1:3]) # ['Java', 'C++']

# Modify
books[2] = "C#" # replace
books.append("Ruby") # add to end
books.insert(1, "Go") # insert at index
books.extend(["SQL","MongoDB"]) # add multiple

# Remove
books.pop() # removes last
books.pop(2) # removes at index 2
books.remove("Go") # removes by value

# Sort & Reverse
books.sort() # alphabetical
books.sort(reverse=True) # reverse alpha
books.reverse() # flip order

# Copy & Clear
copy = books.copy()
books.clear() # empties list

# Nested List (Matrix)
matrix = [[1,2,3],[4,5,6],[7,8,9]]
print(matrix[1][2]) # 6
```

List Comprehension

A shorter, cleaner way to create lists in one line.

```
# Regular way
evens = []
for x in range(1, 11):
if x % 2 == 0:
evens.append(x)

# List comprehension way (same result!)
evens = [x for x in range(1, 11) if x % 2 == 0]
# [2, 4, 6, 8, 10]

odds = [x for x in range(1, 11, 2)]
# [1, 3, 5, 7, 9]
```

2.2 Dictionaries

A dictionary stores data as **key : value** pairs. Keys must be unique.

```
student = {
"id": 12345,
"name": "Bhagya",
"book": "Python Programming"
}

# Access
print(student["name"]) # Bhagya
print(student.get("phone")) # None (no error!)

# Add / Update / Delete
student["fine"] = 50.0 # add new key
student.pop("id") # remove key
del student["book"] # also removes

# Iterate
for key, val in student.items():
print(key, ':', val)

# Useful methods
print(student.keys()) # all keys
print(student.values()) # all values
```

Nested Dictionaries

```
students = {
12345: {"name": "Bhagya", "book": "Python"},
67890: {"name": "Alice", "book": "Java"},
}

# Access nested value
print(students[12345]["name"]) # Bhagya

# Loop through nested dict
for sid, info in students.items():
print('ID:', sid)
for k, v in info.items():
print(k, ':', v)
```

■ **Dictionary Task**

Create a dictionary: book_id, title, author, price

1. Add a key 'available' with value True
2. Increase price by 200
3. Remove the author key
4. Check if book is available
5. Create a multi-book dict and count total books

2.3 Sets

Sets are **unordered** collections of **unique** items. Perfect for removing duplicates or doing math-like operations.

```
# Create a set
emails = {"a@g.com", "b@g.com", "a@g.com"}
print(emails) # duplicate removed!

# Add / Remove
emails.add("c@g.com")
emails.update(["d@g.com", "e@g.com"])
emails.remove("b@g.com") # error if not found
emails.discard("z@g.com") # safe — no error
emails.pop() # removes random item

# Set Operations
a = {1, 2, 3, 4}
```

```
b = {1, 6, 7, 8}
print(a.union(b)) # {1,2,3,4,6,7,8}
print(a.intersection(b)) # {1}
print(a.difference(b)) # {2,3,4}
print(a.symmetric_difference(b)) # {2,3,4,6,7,8}

# Remove duplicates from list using set
ages = [20, 24, 26, 20, 23, 24]
unique_ages = set(ages)
```

2.4 Map, Filter & Reduce

```
from functools import reduce

prices = [100, 200, 300]

# map — transform each item
updated = list(map(lambda x: x + 50, prices))
# [150, 250, 350]

# filter — keep matching items
expensive = list(filter(lambda x: x > 150, updated))
# [250, 350]

# reduce — combine all into one
total = reduce(lambda x, y: x + y, prices)
# 600

# Chaining all three:
books = [
{"title": "Python", "price": 500, "available": True},
{"title": "Java", "price": 800, "available": False},
{"title": "AI", "price": 1200,"available": True},
]
available = list(filter(lambda x: x['available'], books))
updated = list(map(lambda x: x['price']+100, available))
total = reduce(lambda x,y: x+y, updated)
```

2.5 Functions

```
# Basic function
def greet(name):
```

```
print(f"Hello {name}!")

# Default parameter
def greet(name="Guest"):
print("Welcome", name)

# Return value
def calculate_fine(days):
return days * 10

# Keyword arguments
def show_info(name, age):
print(name, age)
show_info(age=21, name='Alice') # order doesn't matter

# *args — variable number of positional args
def total(*numbers):
return sum(numbers)
total(10, 20, 30, 40) # 100

# **kwargs — variable keyword args
def student(**details):
print(details.get('name'), details.get('age'))
student(name='Bob', age=22, course='Python')
```

■ ***args vs **kwargs — Interview Favourite!**

- *args → collects extra positional arguments as a tuple
- **kwargs → collects extra keyword arguments as a dictionary
- def info(*args, **kwargs): print(args, kwargs)
- info("Alice", 20, city="Chennai") → ("Alice",20) {city:"Chennai"}

Higher-Order Functions & Nested Functions

```
# Higher-order: pass function as argument
def execute(operation):
operation()

def search_book(): print('Searching book...')
def search_author(): print('Searching author...')

execute(search_book)
execute(search_author)

# Nested / Inner functions
```

```
def membership_access(type):
def gold(): print('Gold member!')
def silver(): print('Silver member!')
if type == 'gold': return gold()
elif type == 'silver': return silver()
```

2.6 Modules & Packages

```
# books.py (your module)
books = []
def add_book(name): books.append(name)
def show_books():
for b in books: print(b)

# main.py (using your module)
import books
books.add_book('Python')

# Import specific function
from books import show_books

# Import everything
from books import *

# Packages: folder with __init__.py
from student.studentinfo import add_student
from .fine import * # intra-package import
```

■ __name__ == '__main__'

• When you run a file directly, __name__ is '__main__'

• When imported by another file, __name__ is the filename

• Use this to prevent auto-execution when importing:

• if __name__ == '__main__':

• prepare_food()

DAY 3

Modules, Files & Exceptions

File Handling · Exception Handling · User-Defined Exceptions · Pickle

3.1 File Handling

Mode	Meaning	Creates file?	Overwrites?
"r"	Read only	No	N/A
"w"	Write (overwrite)	Yes	Yes
"a"	Append	Yes	No
"rb"	Read binary	No	N/A
"wb"	Write binary	Yes	Yes

```
# Writing to a file
with open("sample.txt", "w") as file:
file.write("Hello, Python!")

# Reading from a file
with open("sample.txt", "r") as file:
data = file.read() # entire file
# OR
lines = file.readlines() # list of lines

# Append mode
with open("log.txt", "a") as file:
file.write("New entry\n")

# Check if file exists
import os
if os.path.exists("sample.txt"):
print("Found!")
os.remove('sample.txt') # delete

# File cursor control
f = open("data.txt", "r")
f.seek(0) # go to start
```

```
f.tell() # current position
```

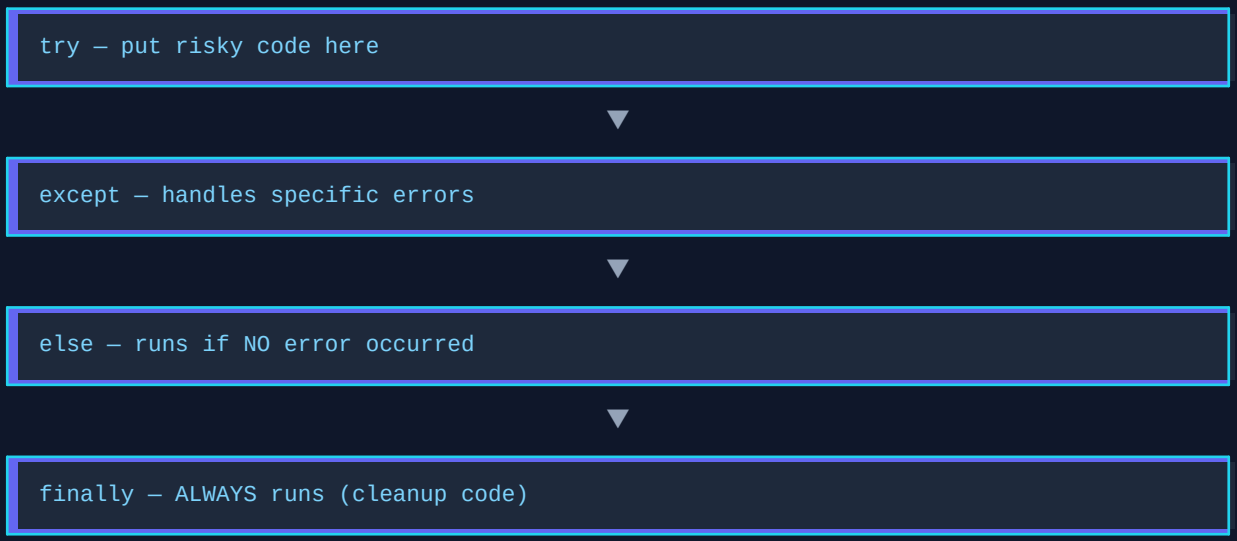
Binary Files & Pickle

```
# Copy an image
with open("photo.jpg","rb") as src:
content = src.read()
with open("copy.jpg","wb") as dst:
dst.write(content)

# Pickle: save Python objects to binary file
import pickle
student = {'name': 'Alice', 'mark': 90}

# Save
with open("data.dat","wb") as f:
pickle.dump(student, f)

# Load
with open("data.dat","rb") as f:
data = pickle.load(f)
print(data)
```

3.2 Exception Handling

Exceptions are runtime errors. Python lets you **catch and handle** them gracefully using try/except.

Exception Flow




```
try:
num = int(input("Enter a number: "))
```

```
print(100 / num)

except ZeroDivisionError:

print("Cannot divide by zero!")

except ValueError:

print("Please enter a valid integer")

except Exception as e:

print('Error:', type(e).__name__)

else:

print("No errors, all good!")

finally:

print("Thank you for visiting!")
```

User-Defined Exceptions

```
class FineNotPaidError(Exception):

pass

try:

status = input("Fine paid? (yes/no): ")

if status == "yes":

print("Book issued")

elif status == "no":

raise FineNotPaidError(

"Clear fine before issuing book"

)

except FineNotPaidError as e:

print(e)
```

Standard Python Exceptions

Exception	When it occurs
ZeroDivisionError	Dividing by zero
ValueError	Wrong value type e.g. int('abc')
TypeError	Wrong type e.g. 'age' + 5
FileNotFoundError	File doesn't exist
IndexError	List index out of range

KeyError	Dictionary key not found
PermissionError	Access denied
NameError	Variable not defined

■ Exception Task

Book Issue System:

1. Accept student ID (should be integer)

2. If invalid input → print error message

3. If valid → print 'Book issued successfully'

4. In finally → always print 'Visit Again!'

DAY 4

Object-Oriented Programming (OOPs)

Classes · Inheritance · Polymorphism · Encapsulation · Abstraction

4.1 Classes & Objects

A **class** is a blueprint. An **object** is a real instance created from that blueprint.

```
class Book:
library_name = "Smart Library" # class variable

def __init__(self, id, title, author, price):
# Instance variables
self.id = id
self.title = title
self.author = author
self.price = price

def display_info(self): # instance method
print(f"[{self.id}] {self.title} by {self.author}")

@classmethod
def show_library(cls): # class method
print(f"Welcome to {cls.library_name}")

@staticmethod
def calculate_gst(amount): # static method
return amount * 0.05

# Create objects
python = Book("B101", "Python Prog.", "Guido", 25.99)
python.display_info()
Book.show_library()
Book.calculate_gst(200)
```

Method Type	Decorator	First Param	Access
Instance Method	None	self	Instance & class vars
Class Method	@classmethod	cls	Class vars only

Static Method	@staticmethod	None (no self/cls)	Neither — utility function

4.2 Constructor & Destructor

```
class Library:
def __init__(self, name): # Constructor
self.name = name
print(f"Library {name} created")

def __del__(self): # Destructor
print(f"Library {self.name} destroyed")

lib = Library("Smart Library") # __init__ called
del lib # __del__ called
```

4.3 Inheritance

Inheritance lets a child class **reuse code** from a parent class. Python supports 5 types.

Type	Structure	Meaning	Airport Example
Single	A → B	One parent, one child	Person → Student
Multilevel	A → B → C	Chain of inheritance	Person → Student → VIPStudent
Multiple	A+B → C	Multiple parents	SecurityCheck + ImmigrationCheck → Passenger
Hierarchical	A → B, A → C	One parent, many children	Person → Student, Person → Librarian
Hybrid	Mix of above	Combination	Complex systems

```
# Single Inheritance
class Person:
def __init__(self, name, email):
self.name = name
self.email = email
def login(self):
return f"{self.name} logged in"

class Student(Person):
```

```
def __init__(self, name, email, sid):
super().__init__(name, email) # call parent
self.student_id = sid

# Multiple Inheritance
class SecurityCheck:
def security_check(self): return 'Security done'

class ImmigrationCheck:
def immigration_check(self): return 'Immigration done'

class Passenger(Person, SecurityCheck, ImmigrationCheck):
pass # inherits ALL methods from all parents
```

4.4 Polymorphism

Polymorphism means the **same method name behaves differently** in different classes.

```
# Method Overriding (Runtime Polymorphism)
class Notification:
def send(self):
return "Sending notification"

class EmailNotification(Notification):
def send(self):
return "Sending EMAIL"

class SMSNotification(Notification):
def send(self):
return "Sending SMS"

for notif in [Notification(), EmailNotification(), SMSNotification()]:
print(notif.send()) # different output, same method!

# Method Overloading (Python uses default params)
class SearchBooks:
def search(self, title=None, author=None):
if title and author:
print(f'Search: {title} by {author}')
elif title:
print(f'Search by title: {title}')
elif author:
print(f'Search by author: {author}')
```

4.5 Encapsulation

Encapsulation means **hiding data** inside a class. You control what can be accessed from outside.

Access Level	Prefix	Accessible From
Public	name	Anywhere
Protected	_name	Class & subclasses (convention only)
Private	__name	Only inside the class — name mangling

```
class Book:
def __init__(self, title, author, price):
self.title = title
self.__author = author # private
self.price = price

def get_author(self): # getter
return self.__author

def set_author(self, author): # setter
self.__author = author

book = Book('Python', 'Guido', 25.99)
# print(book.__author) # ERROR!
print(book.get_author()) # Guido
print(book._Book__author) # name mangling access
```

4.6 Abstraction

Abstraction means showing **only what's necessary** and hiding how it works. Use abstract classes to enforce a contract.

```
from abc import ABC, abstractmethod

class Library(ABC): # abstract base class
@abstractmethod
def issue_book(self): # MUST be implemented
pass

@abstractmethod
def return_book(self):
pass
```

```
def display_name(self): # concrete method
print("Smart Library")

class CollegeLibrary(Library):
def issue_book(self):
print("Issued to college student")
def return_book(self):
print("Returned")

# Library() ← ERROR! Can't instantiate abstract class
clib = CollegeLibrary() # OK
clib.issue_book()
```

4.7 Composition & Aggregation

Relationship	Meaning	Dependency
Composition	Class A contains Class B, B cannot exist without A	Strong — B is created inside A
Aggregation	Class A uses Class B, B can exist independently	Weak — B is passed into A

```
class Book:
def __init__(self, title): self.title = title

class Student:
def __init__(self, name): self.name = name

class Library:
def __init__(self, student):
# Composition: Book lives inside Library
self.book = Book("Python Programming")
# Aggregation: Student exists independently
self.student = student

lib = Library(Student("Alice"))
print(lib.book.title) # Python Programming
print(lib.student.name) # Alice
```

4.8 Magic Methods & Operator Overloading

```
class Library:
def __init__(self, name):
self.name = name

def __str__(self): # print(lib) calls this
return f"Library: {self.name}"

def __len__(self): # len(lib) calls this
return len(self.name)

def __add__(self, other): # lib1 + lib2
return self.name + " & " + other.name

def __eq__(self, other): # lib1 == lib2
return self.name == other.name

def __del__(self): # destructor
print(f"{self.name} destroyed")

lib1 = Library("Smart Library")
lib2 = Library("City Library")
print(lib1) # Library: Smart Library
print(len(lib1)) # 13
print(lib1 + lib2) # Smart Library & City Library
print(lib1 == lib2) # False
```

DAY 5

Advanced Python

Collections · Regex · Decorators · Generators · Context Managers · DB · Best Practices

5.1 Collections Module

The **collections** module gives you specialized, powerful container types beyond list/dict/set.

```
from collections import Counter, defaultdict, namedtuple, deque, OrderedDict

# Counter: count occurrences
data = ['Python','Java','Python','C++','Python']
counter = Counter(data)
print(counter) # Counter({'Python':3,...})
print(counter.most_common(1)) # [('Python', 3)]

# defaultdict: default value for missing keys
d = defaultdict(int) # default is 0
d['Python'] += 1
print(d['NewKey']) # 0 (no KeyError!)

# namedtuple: tuple with field names
Book = namedtuple('Book', ['title','author','price'])
b1 = Book('Python', 'Guido', 25.99)
print(b1.title) # Python

# deque: double-ended queue
q = deque()
q.append('Task1') # add right
q.appendleft('Task0') # add left
q.pop() # remove right
q.popleft() # remove left

# OrderedDict: remembers insertion order
od = OrderedDict()
od['a'] = 1
od['b'] = 2
```

5.2 Regular Expressions (Regex)

Regular expressions let you **search, validate, and extract patterns** from strings using the **re** module.

Function	What it does
re.match(pattern, str)	Match at the BEGINNING of string
re.fullmatch(pattern, str)	Match the ENTIRE string
re.search(pattern, str)	Find first match ANYWHERE in string
re.findall(pattern, str)	Find ALL matches — returns list
re.sub(pattern, repl, str)	Replace matches with something else

```
import re

# Validate email
result = re.fullmatch(r"\w+@airport\.com", "john@airport.com")
print(result is not None) # True

# Validate passport: 1 letter + 7 digits
passports = re.findall(r"[A-Z]\d{7}", "A1234567 and B7654321")
print(passports) # ['A1234567', 'B7654321']

# Detect dangerous words
found = re.search(r"\b(bomb|gun|knife)\b", "bag has a knife")
print(found is not None) # True

# Mask credit card
masked = re.sub(r"(\d{4})\d{8}(\d{4})", r"\1****\2",
"Card: 4567891234567890")
print(masked) # Card: 4567****7890

# Validate Indian mobile (starts with 8 or 9)
valid = re.fullmatch(r"[89]\d{9}", "9876543210")
```

Regex Quick Reference

Pattern	Matches
\d	Any digit [0-9]
\w	Word character [a-z A-Z 0-9 _]
\s	Whitespace
\b	Word boundary

.	Any character except newline
+	One or more
*	Zero or more
?	Zero or one
{n}	Exactly n times
{n,m}	Between n and m times
^	Start of string
$	End of string

5.3 Decorators

A decorator is a function that **wraps another function** to add behaviour before/after it runs — without changing the original code.

```
# Basic Decorator
def security_check(func):
def wrapper(*args, **kwargs):
print("Performing security check...")
func(*args, **kwargs)
print("Check done.")
return wrapper

@security_check
def enter_lounge():
print("Welcome to VIP Lounge")

enter_lounge()
# Output:
# Performing security check...
# Welcome to VIP Lounge
# Check done.

# Stacking decorators (applied bottom-up)
@security_check
@baggage_check
@passport_check
def board_flight(name):
```

```
print(name, "boarded the flight")
```

5.4 Generators

Generators produce values **one at a time on demand** using **yield**. Unlike lists, they don't store everything in memory — great for large data.

```
def passengers():
yield "John" # pauses here, returns "John"
yield "Alice" # resumes here next call
yield "Bob"

p = passengers()
print(next(p)) # John
print(next(p)) # Alice
print(next(p)) # Bob

# Generator for Fibonacci sequence
def fibonacci(n):
a, b = 0, 1
for _ in range(n):
yield a
a, b = b, a + b

for num in fibonacci(10):
print(num, end=' ') # 0 1 1 2 3 5 8 13 21 34
```

5.5 Context Managers

Context managers use the **with** keyword to automatically set up and clean up resources — no need to manually close files or connections.

```
# File handling — file auto-closes after block
with open("data.txt", "w") as f:
f.write("Hello!")
# File is closed here automatically

# Custom context manager
class SecurityScanner:
def __enter__(self):
print("Scanner Activated")
return self
```

```
def __exit__(self, exc_type, exc_val, tb):
print("Scanner Deactivated")

with SecurityScanner() as scanner:
name = input("Passenger name: ")
print(f"Passenger {name} Cleared")
```

5.6 Database Connectivity

Python can connect to SQL Server, MySQL, SQLite and more using database drivers.

```
import pyodbc

# Connect to SQL Server
conn = pyodbc.connect(
"DRIVER={ODBC Driver 17 for SQL Server};"
"SERVER=(localdb)\\MSSQLLocalDB;"
"DATABASE=AirportDb;"
"Trusted_Connection=yes;"
)
cursor = conn.cursor()

# CREATE
cursor.execute('''CREATE TABLE Passengers(
ID INT PRIMARY KEY, Name NVARCHAR(100),
PassportNumber NVARCHAR(20))''')

# INSERT
cursor.execute("INSERT INTO Passengers VALUES(1,'Alice','P123456')")

# SELECT
cursor.execute('SELECT * FROM Passengers')
for row in cursor.fetchall(): print(row)

# UPDATE
cursor.execute("UPDATE Passengers SET Name='Bob' WHERE ID=1")

# DELETE
cursor.execute('DELETE FROM Passengers WHERE ID=1')
conn.close()
```

5.7 Best Practices & PEP 8

Rule	Example
Variables/Functions: snake_case	student_name, calculate_fine()
Classes: PascalCase	StudentRecord, AirportSystem
Constants: UPPER_SNAKE_CASE	MAX_WEIGHT, MIN_AGE
Indentation: 4 spaces	def foo():■....pass
Max line length: 79 chars	Break long lines with backslash or brackets
Imports at top, one per line	import os■import datetime
Spaces around operators	x = 5 + 3 not x=5+3

■ Coding Best Practices

- 1. Use meaningful variable/function/class names
- 2. Write clean, readable, self-documenting code
- 3. Follow DRY — Don't Repeat Yourself
- 4. Follow SRP — Single Responsibility Principle
- 5. Handle exceptions properly
- 6. Use generators instead of lists for large data
- 7. Close resources properly (files, DB connections)
- 8. Use comments only when needed — don't over-comment
- 9. Avoid unnecessary loops and nested loops
- 10. Use built-in functions — they're optimized!

DAY 6

Python Unit Testing

STLC · Testing Types · unittest · Fixtures · Parameterized Tests

6.1 Software Testing Concepts

STLC — Software Testing Life Cycle

Phases of STLC

1. Requirement Analysis
▼
2. Test Planning
▼
3. Test Case Development
▼
4. Environment Setup
▼
5. Test Execution
▼
6. Bug Reporting
▼
7. Test Closure

Types of Testing

Type	What is tested	Who tests
Unit Testing	Individual functions in isolation	Developer

Integration Testing	Combined modules working together	Dev / QA
System Testing	Complete application end-to-end	QA Team
Acceptance Testing	Client validates before launch	Client / Business
Performance Testing	Speed and stability under load	Performance Team

Testing Techniques

Technique	What tester sees	Best for
Black Box	Only inputs and outputs	UI-based, functional testing
White Box	Full source code	Code logic, unit tests
Grey Box	Partial code knowledge	Integration testing

6.2 unittest — Python's Built-in Testing Framework

```
import unittest

def calculate_fine(days):
return days * 10

def issue_book(available):
if available: return "Book issued"
raise Exception("Book not available")

class LibraryTests(unittest.TestCase):

def test_calculate_fine(self):
self.assertEqual(calculate_fine(5), 50)
self.assertEqual(calculate_fine(0), 0)
self.assertEqual(calculate_fine(10), 100)

def test_issue_book(self):
self.assertEqual(issue_book(True), "Book issued")
with self.assertRaises(Exception):
issue_book(False)

if __name__ == '__main__':
unittest.main()
```

Common Assertions

Method	What it checks
assertEqual(a, b)	a == b
assertNotEqual(a, b)	a != b
assertTrue(x)	x is True
assertFalse(x)	x is False
assertRaises(Error, fn, arg)	fn(arg) raises Error
assertIn(item, list)	item is in list

Test Fixtures: setUp & tearDown

```
class LibraryTests(unittest.TestCase):

def setUp(self):

print("Setting up...")

self.book_available = True # runs before EACH test

def tearDown(self):

print("Cleaning up...")

# runs after EACH test — close connections, reset state

def test_issue_book(self):

result = issue_book(self.book_available)

self.assertEqual(result, "Book issued")
```

Parameterized Tests with subTest

```
class LibraryTests(unittest.TestCase):

def test_calculate_fine(self):

test_data = [(5, 50), (0, 0), (10, 100)]

for days, expected in test_data:

with self.subTest(days=days):

self.assertEqual(calculate_fine(days), expected)
```

■ Running Tests from Command Line

- Run all tests: python -m unittest test_file.py
- Run specific test: python -m unittest -k test_calculate_fine
- Run test suite: use unittest.TestSuite()

- Verbose output: python -m unittest -v test_file.py

PROJECT

Airport Security Management System

Full OOPs Implementation — 21 Tasks Covered

Project Overview

This is the major OOPs project from your training. It implements a complete Airport Security Management System covering all OOPs concepts from Task 1 to Task 21.

Classes in the System

Class	Inherits From	Key Features
Person	—	login(), logout()
Passenger	Person, SecurityCheck, ImmigrationCheck	verify_passport(), scan_baggage(), board_flight()
VIPPassenger	Passenger	priority_checkin(), lounge_access()
DomesticPassenger	Passenger	security_check() — domestic version
InternationalPassenger	Passenger	security_check() — international version
Flight	—	flight_id, flight_name, destination
Airport	—	Class var: airport_name, show_airport_name()
Baggage	—	calculate_extra_baggage_fee() static method
SecurityOfficer	Person, SecurityProcess(ABC)	verify_identity(), baggage_scan()
SecurityProcess	ABC	Abstract: verify_identity(), baggage_scan()
SecuritySystem	—	Aggregation — uses existing Passenger objects

Key Code Highlights

```
# Task 7: Static method for extra baggage fee
@staticmethod
def calculate_extra_baggage_fee(weight):
if weight > 20:
extra = weight - 20
return f"Extra fee: Rs.{extra * 500}"
return "Baggage within limit"

# Task 14: Encapsulation — private passport
def set_passport(self, no): self.__passport_number = no
def get_passport(self): return self.__passport_number

# Task 19: Magic method __str__
def __str__(self):
return self.passenger_name

# Task 20: Operator overloading — add baggage weights
def __add__(self, other):
return self.weight + other.weight

b1 = Baggage(1, 25)
b2 = Baggage(2, 15)
print(b1 + b2) # 40

# Task 21: Exception handling
try:
p = Passenger(2, 'Bob', '123456', -15)
except ValueError as e:
print(e) # Negative baggage weight not allowed
# OR Invalid passport number
```

Regex Tasks Summary

Task	Pattern	Example Match
Passenger ID	PSG\d{4}	PSG1234
Flight Number	[A-Z]{2}\d{3,4}	AI203
Passport Number	[A-Z]\d{7}	A1234567
Airport Email	\w+@airport\.com	john@airport.com

Boarding Gate	GATE-\d{1,2}	GATE-7
Time Format	([01]\d|2[0-3]):[0-5]\d	09:45
Suspicious Bag Tag	SB-\d{5}	SB-12345
Runway Code	RWY-\d{2}[A-Z]	RWY-27A
Credit Card Mask	(\d{4})\d{8}(\d{4})	\1****\2

Quick Reference Cheat Sheet

Python Keywords to Know

`False, None, True`	`and, or, not`	`if, elif, else`
`for, while, break, continue`	`def, return, lambda`	`class, self, super()`
`try, except, finally, raise`	`import, from, as`	`with, yield, pass`
`in, is, not in, is not`	`@classmethod, @staticmethod`	`@abstractmethod, @property`

OOPs Concepts at a Glance

Concept	One-line Definition
Class	Blueprint / template for creating objects
Object	Real instance created from a class
Constructor (__init__)	Runs automatically when object is created
Encapsulation	Hiding data using private variables + getters/setters
Inheritance	Child class reuses code from parent class
Polymorphism	Same method name, different behaviour per class
Abstraction	Show only what's needed, hide implementation
Composition	Class contains another class — strong dependency
Aggregation	Class uses another class — weak dependency
Decorator	Wraps a function to add behaviour
Generator	Produces values one at a time using yield

You made it through all 6 days! Keep coding ■
Python Training Notes — Personal Textbook